



Large Language Models & Knowledge Representation

Luana Bulla

University of Catania • ISTC-CNR

luana.bulla@phd.unict.it • luana.bulla@istc.cnr.it

Prompting Engineering

Zero-Shot Prompting

No Examples Required

The model receives direct instructions without any demonstrations or sample responses to guide its behavior.

Immediate Task Performance

The prompt tells the model exactly what to do, relying on its pre-trained knowledge to complete the task.

Example: "Classify the text into neutral, negative or positive.

Text: I think the vacation is okay. Sentiment: *Neutral*"



Prompting Engineering

Few-Shot Prompting

Learning from Examples

Few-shot prompting leverages in-context learning by providing demonstrations within the prompt to guide model behavior and improve performance.

These demonstrations serve as conditioning for subsequent examples, helping the model understand the expected response format and pattern.

Example:

This is awesome! // Positive |

This is bad! // Negative |

Wow that movie was rad! // Positive |

What a horrible show! // ... **R: Negative**



Few-Shot Best Practices

Label Distribution Matters

Both the label space and distribution of input text in demonstrations significantly impact performance, regardless of individual label correctness.

Format is Critical

Consistent formatting plays a key role in performance. Even random labels perform better than no labels at all.

Natural Distribution Helps

Selecting random labels from the true distribution of labels outperforms uniform distribution selection.

Limitations

Unable to perform complex reasoning tasks such as common-sense reasoning questions.

Prompting Engineering

Chain-of-Thought (CoT) Prompting

01

Step-by-Step Reasoning

Forces LLMs to output intermediate steps leading to the final answer, breaking complex problems into manageable parts.

02

Enhanced Problem-Solving

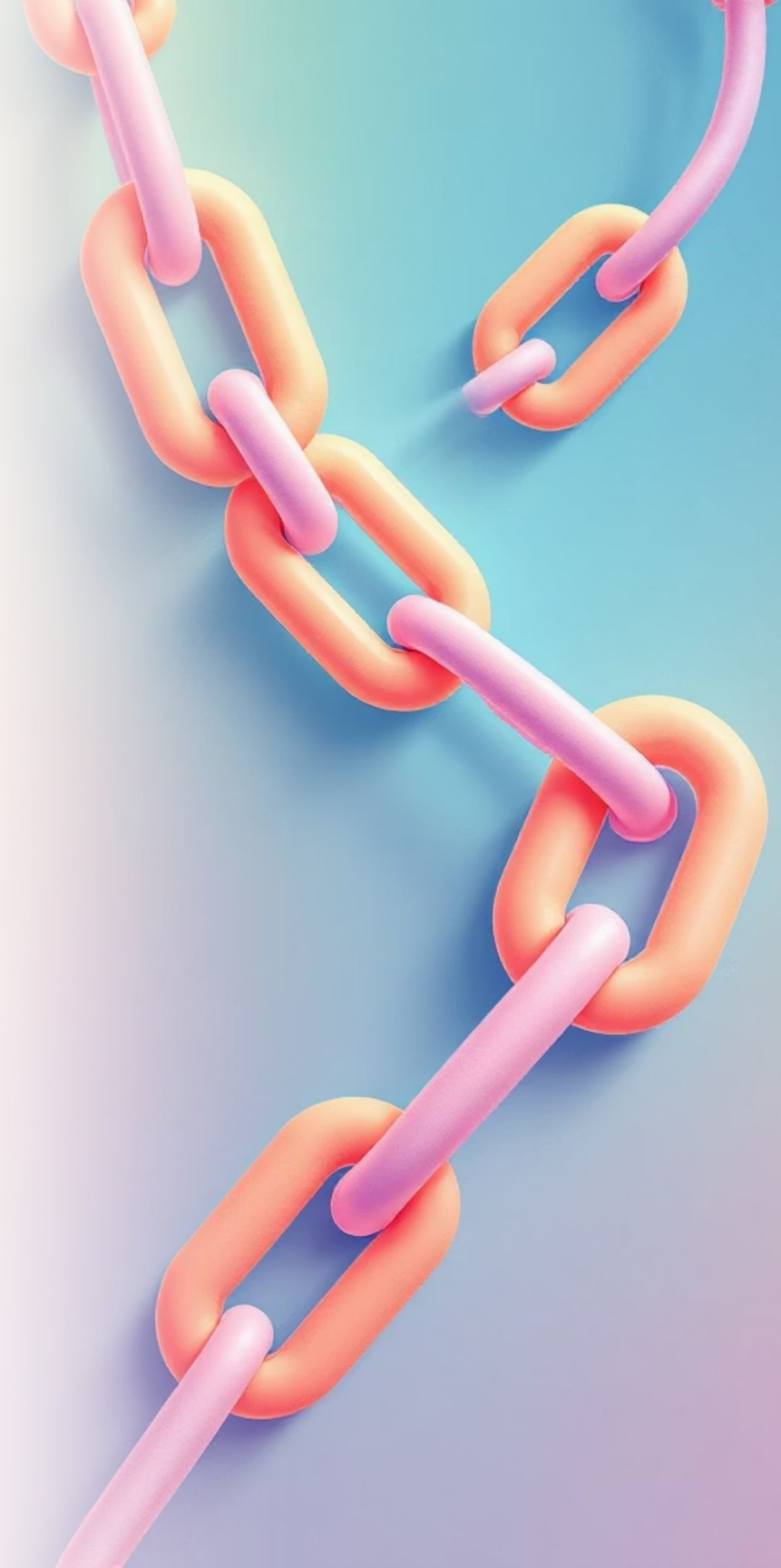
Allows models to focus on one step at a time rather than considering the entire problem simultaneously.

03

Interpretable Results

Provides transparency into model behavior, showing the reasoning path from question to answer.

Particularly effective for complex multi-step problems that would be difficult to solve in a single inference step.



Chain-of-Thought in Action

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Zero-Shot Chain-of-Thought

By simply adding "Let's think step by step" to the prompt, we can trigger chain-of-thought reasoning without providing explicit examples, combining the benefits of both approaches.

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?
A:

(Output) The answer is 8. ✗

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?
A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?
A: The answer (arabic numerals) is

(Output) 8 ✗

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?
A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

LLM Generation Parameters



Temperature

Regulates randomness in text generation. Higher values (0.7-1.0) produce creative, diverse outputs. Lower values (0.1-0.3) generate predictable, coherent text.



Repetition Penalty

Discourages token repetition within generated text, preventing redundant or cyclical outputs that reduce content quality.



Max New Tokens

Limits the maximum number of tokens generated, controlling output length and computational resources during inference.



LLM Landscape Overview

OpenAI GPT Family

GPT-3.5 (175B Parameters)

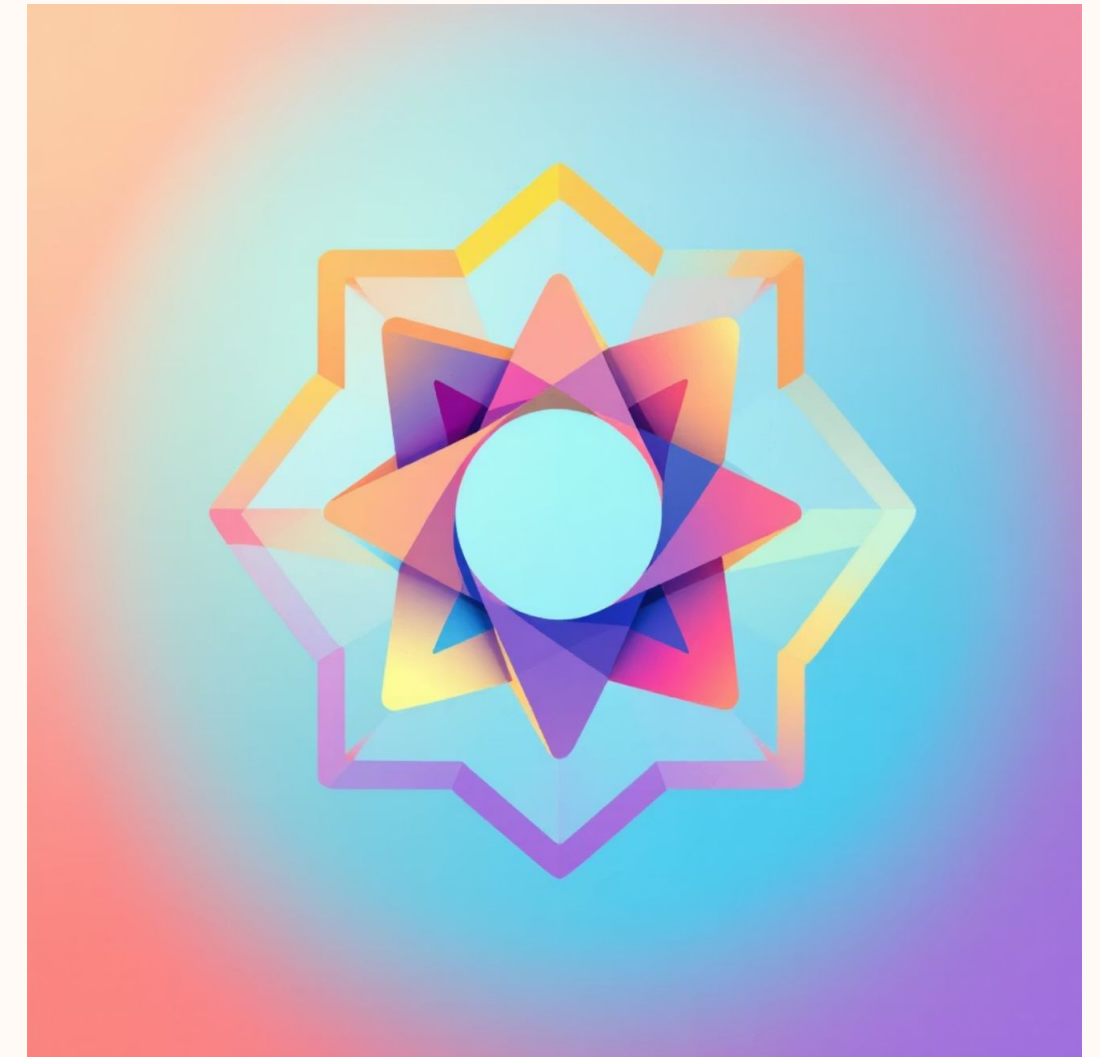
- Generates realistic and creative text
- Performs linguistic translation
- Provides informative question answering

GPT-4 (Est. 1.7T Parameters)

- Advanced reasoning and problem-solving
- Enhanced natural language understanding
- Multimodal capabilities (text, images, code)

GPT-4o

Latest iteration with optimized performance and expanded capabilities across multiple domains.





Meta LLaMA3 Family



LlamaChat Enhancement

Chat variant trained on over 1 million human annotations, refined using RLHF with rejection sampling and PPO.



LLaMa Small (8B)

Efficient model for resource-constrained environments while maintaining strong performance.



LLaMa Medium (70B)

Balanced performance and computational requirements for most applications.



LLaMa Large (405B)

State-of-the-art performance with massive parameter count for complex reasoning tasks.

LLM Landscape Overview

Mistral Family

Mixtral 8x7B: Uses sparse Mixture of Experts (MoE) architecture with 8 distinct expert groups, totaling 46.7B parameters.

Mistral 7B: Outperforms LLaMa 13B using Grouped-query attention (GQA) and Sliding Window Attention (SWA).

DeepSeek Family

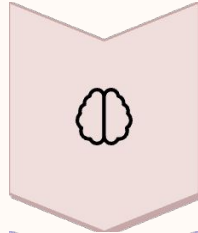
Generalist LLM alternative to LLaMa2/3 and GPT-3.5. The 7B version competes with StarCoder, while 33B focuses on reasoning.

Features: 87% code dataset, RLHF optimization, vision-language capabilities, multimodal processing.



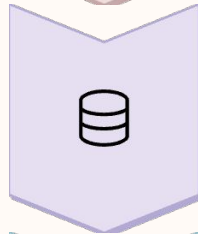
The LLM-KG Synergy

"The synergy between LLMs and KGs opens new frontiers for AI applications, enhancing machines' ability to process, interpret, and generate information with context and accuracy"



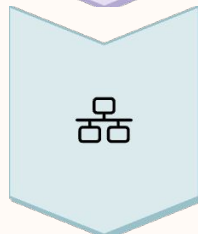
LLMs for KGs

How LLMs bolster and refine Knowledge Graph functionality



KGs-enhanced LLMs

KGs leverage to enhance LLM capabilities and accuracy



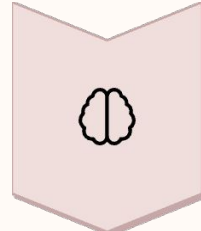
LLM-KG Cooperation

Collaborative synergy achieving advanced, complex outcomes



The LLM-KG Synergy

"The synergy between LLMs and KGs opens new frontiers for AI applications, enhancing machines' ability to process, interpret, and generate information with context and accuracy"



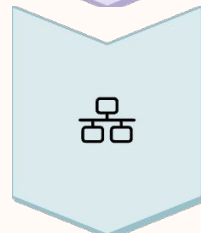
LLMs for KGs

How LLMs bolster and refine Knowledge Graph functionality



KGs-enhanced LLMs

KGs leverage to enhance LLM capabilities and accuracy



LLM-KG Cooperation

Collaborative synergy achieving advanced, complex outcomes



LLMs for Knowledge Graphs



KG Construction

Automated generation of knowledge graph structures from unstructured text data



KG-to-Text Generation

Converting structured knowledge into natural language descriptions



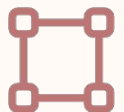
KG Reasoning

Inferring new knowledge from existing graph relationships and patterns



KG Completion

Filling missing entities and relationships in incomplete knowledge graphs



KG Embedding

Creating dense vector representations of entities and relations



KG Validation

Verifying accuracy and consistency of knowledge graph content

LLMs for Knowledge Graphs



KG Construction

Automated generation of knowledge graph structures from unstructured text data



KG-to-Text Generation

Converting structured knowledge into natural language descriptions



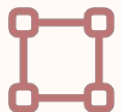
KG Reasoning

Inferring new knowledge from existing graph relationships and patterns



KG Completion

Filling missing entities and relationships in incomplete knowledge graphs



KG Embedding

Creating dense vector representations of entities and relations



KG Validation

Verifying accuracy and consistency of knowledge graph content

LLMs for Knowledge Graphs

KG Construction Tasks

1 Ontology Generation

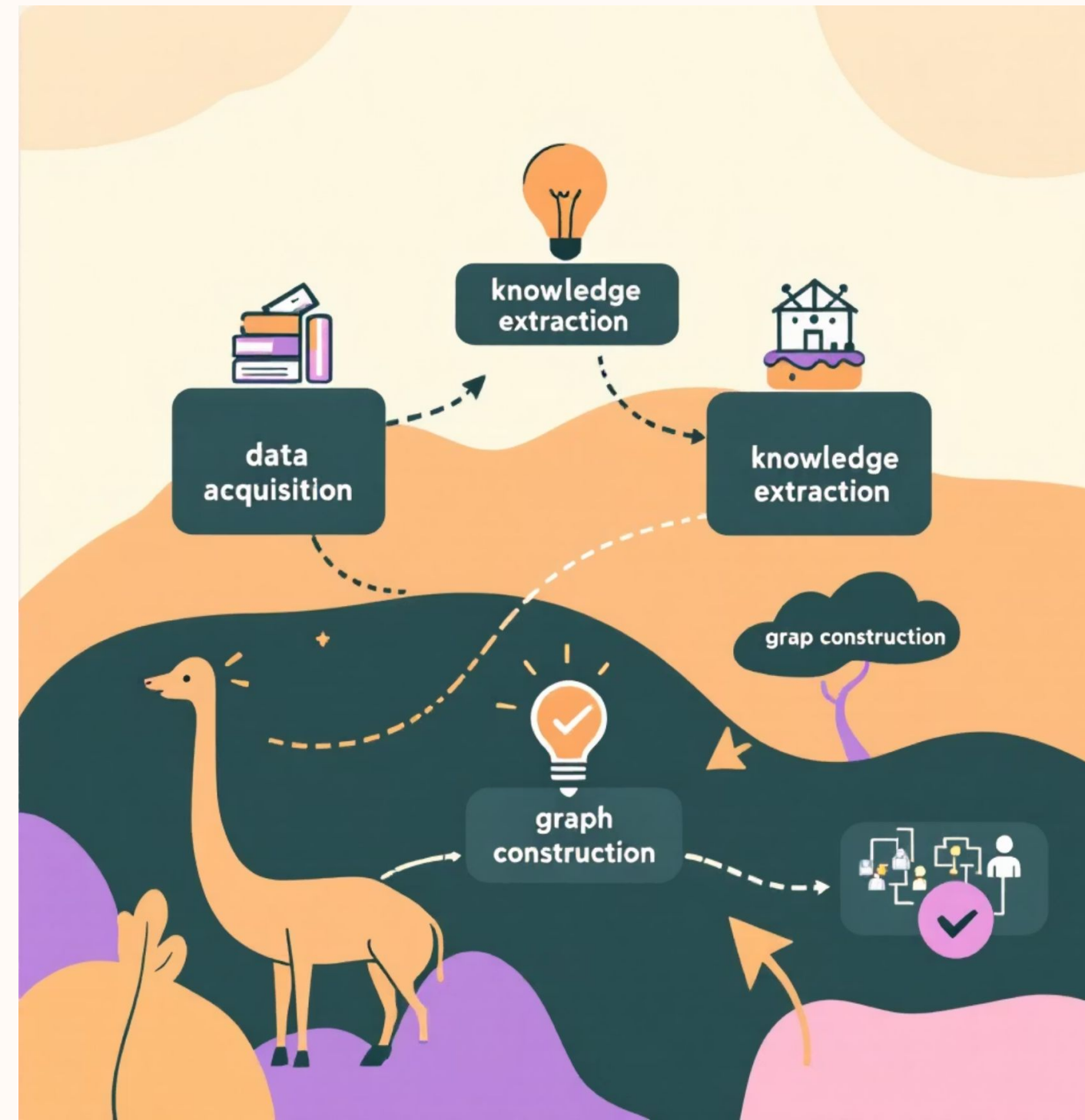
Defining conceptual frameworks and taxonomies

2 Entity Extraction and Alignment

Identifying and aligning entities across data sources

3 Relation Extraction

Discovering semantic relationships between entities



LLMs for Knowledge Graphs

KG Construction Tasks

1 Ontology Generation

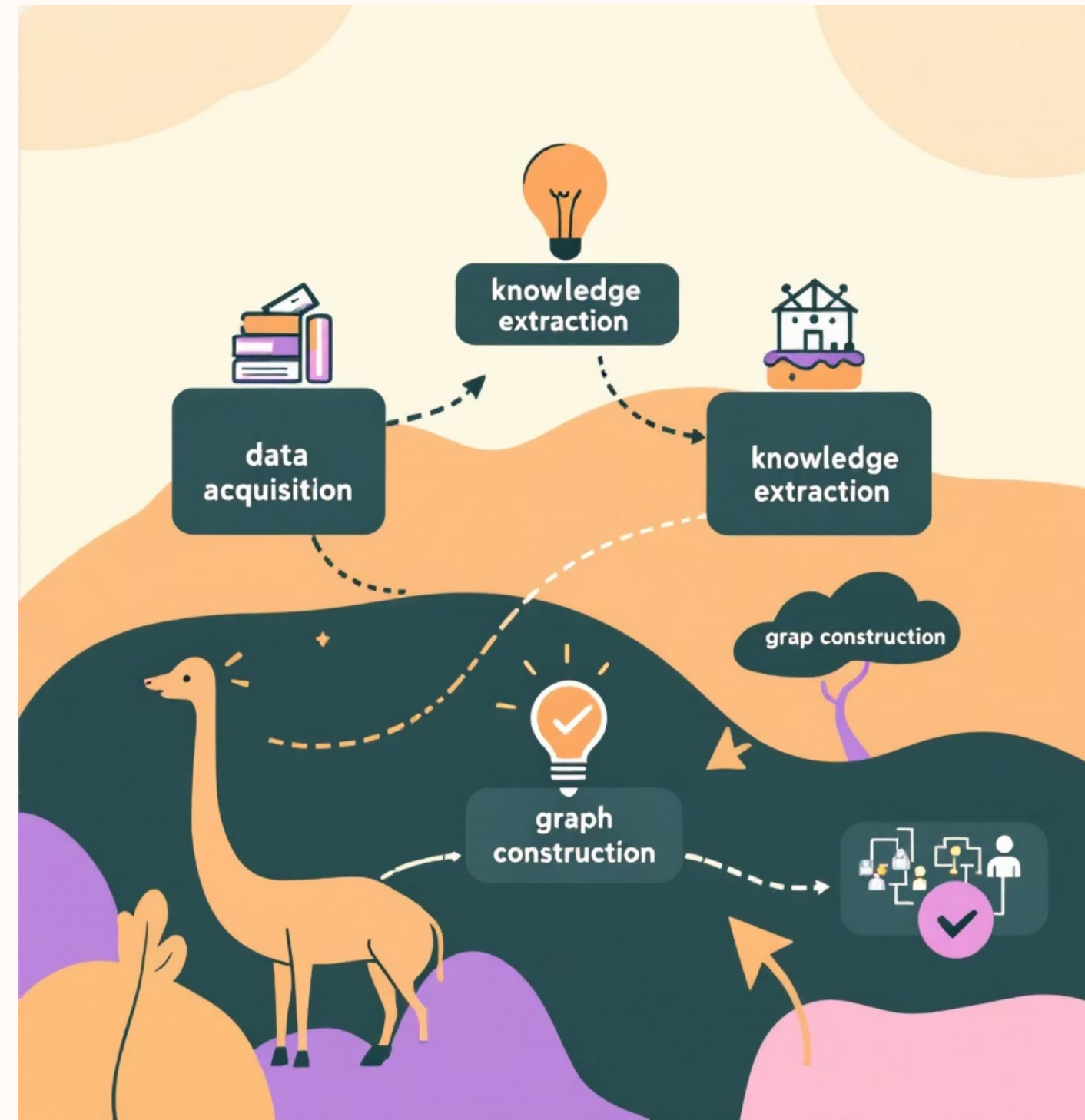
Defining conceptual frameworks and taxonomies

2 Entity Extraction and Alignment

Identifying and aligning entities across data sources

3 Relation Extraction

Discovering semantic relationships between entities



Ontology Generation Process

LLMs contribute to creating comprehensive ontologies through domain-specific fine-tuning, enhancing integration into specialized knowledge areas.

Concept and Relation Extraction Identifying domain-specific concepts and semantic variations	Property Identification Defining attributes and characteristics of entities through fine-tuning LMs	Ontology Enrichment Expanding existing structures with new knowledge
Ontology Alignment Mapping between different ontological frameworks	Ontology Learning Automated acquisition of ontological knowledge	

Current Limitations



LLMs face computational intensity and bias challenges, requiring broader evaluation frameworks.

Ontology Generation Process

Concept and Relation Extraction

Concept Extraction Process

Concept extraction identifies domain-specific concepts and semantic term variations from unstructured text.

Primary goal: Define the vocabulary (classes/concepts) of your ontology through automated text analysis.

Relation Extraction Framework

Relation extraction identifies non-taxonomic relationships between extracted concepts.

Key focus: Building semantic connections that form the backbone of knowledge graph structures.

Example

"The lion (***Panthera leo***) is a ***large*** feline of the ***Felidae*** family. It lives in ***Africa and India***. The male has a thick ***golden mane***."

- hasSpecies » Panthera leo
- hasFamily » Felidae
- liveIn » Africa, India
- hasPhysicalCharacteristics » mane
- hasSize » large

This example demonstrates how LLMs automatically extract structured relationships from natural language, transforming unstructured text into actionable knowledge graph triplets.

Relation Extraction vs Property Identification

Understanding the critical distinction between discovering linguistic relations and mapping to formal ontological properties.

Aspect	Relation Extraction	Property Identification
Input	Natural language text	Extracted relation or semantic description
Output	Triple with generic relation	Standard RDF/OWL property from ontologies
Goal	Discover implicit relations	Map to known/formal property
Focus	Linguistic understanding	Ontological consistency

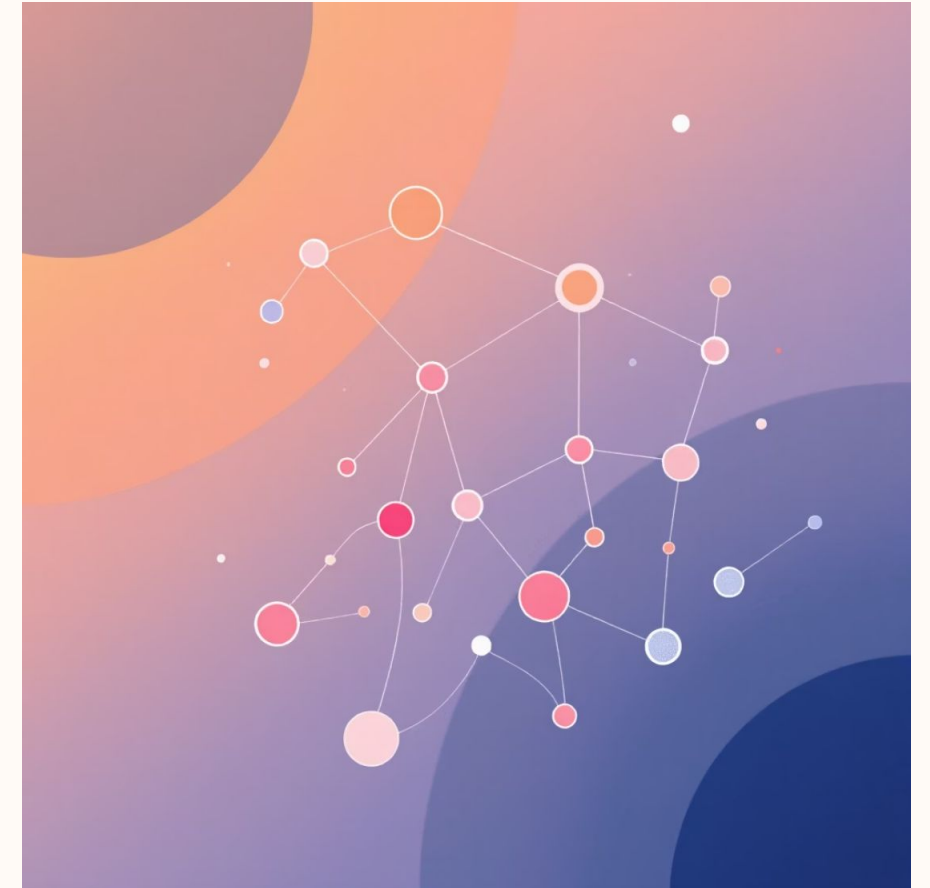
Fine-tuned LLMs excel at pre-annotating data for property identification, reducing manual annotation time while maintaining semantic accuracy.

Ontology Generation Process

Ontology Enrichment

LLMs (often with specialized fine-tuning) enhance existing ontologies by incorporating new information and refining concept definitions.

This approach proves especially valuable in dynamic domains where ontologies must continuously adapt to emerging facts and evolving knowledge structures.





Ontology Generation Process

Ontology Alignment

LLMs facilitate ontology alignment by comprehending and comparing semantic meaning of concepts and relationships across different ontological frameworks.

Advanced Alignment Techniques



Semantic Similarity Matching

Leverages vector embeddings and semantic similarity metrics to identify corresponding concepts across ontologies with high precision.



Zero-shot Prompting

Enables alignment without training data by using carefully crafted prompts to guide LLM reasoning about concept relationships.



Structural Alignment

Injects hierarchical and domain information into prompts, utilizing few-shot techniques to understand complex ontological relationships.

Ontology Generation Process

Ontology Learning from Scratch

Automated construction of new ontologies through comprehensive text analysis. This process identifies relevant concepts, properties, and relationships that characterize specific domains directly from large-scale textual data.



Multi-Step Ontology Learning Pipeline

"Aspirin is a non-steroidal anti-inflammatory drug used to treat pain, fever, and inflammation"

Step	Prompt	Output
1. Concept Extraction	Extract all relevant domain concepts from the text. Return as candidate ontology classes.	<i>Aspirin, Non-steroidal anti-inflammatory drug, Pain, Fever, Inflammation</i>
2. Taxonomy Induction	Identify subclass relationships among extracted terms.	<i>Aspirin $\subseteq$ Non-steroidal anti-inflammatory drug $\subseteq$ Drug</i>
3. Relation Extraction	Extract meaningful semantic relations in Subject-Predicate-Object format.	<i>Aspirin treats Pain; Aspirin treats Fever; Aspirin treats Inflammation</i>
4. OWL Generation	Represent extracted knowledge as formal ontology axioms using OWL syntax.	Formal RDF/OWL triples with class declarations and properties

Multi-Step Ontology Learning Pipeline

```
Class: Drug
Class: NonSteroidalAntiInflammatoryDrug
  SubClassOf: Drug
Class: Aspirin
  SubClassOf: NonSteroidalAntiInflammatoryDrug
Class: Pain, Fever, Inflammation

ObjectProperty: treats
  Domain: Drug
  Range: Symptom

Axiom:
  Aspirin treats Pain
  Aspirin treats Fever
  Aspirin treats Inflammation
```

"Aspirin is a non-steroidal anti-inflammatory drug used to treat pain, fever, and inflammation."

LLMs for Knowledge Graphs

KG Construction Tasks

1 Ontology Generation

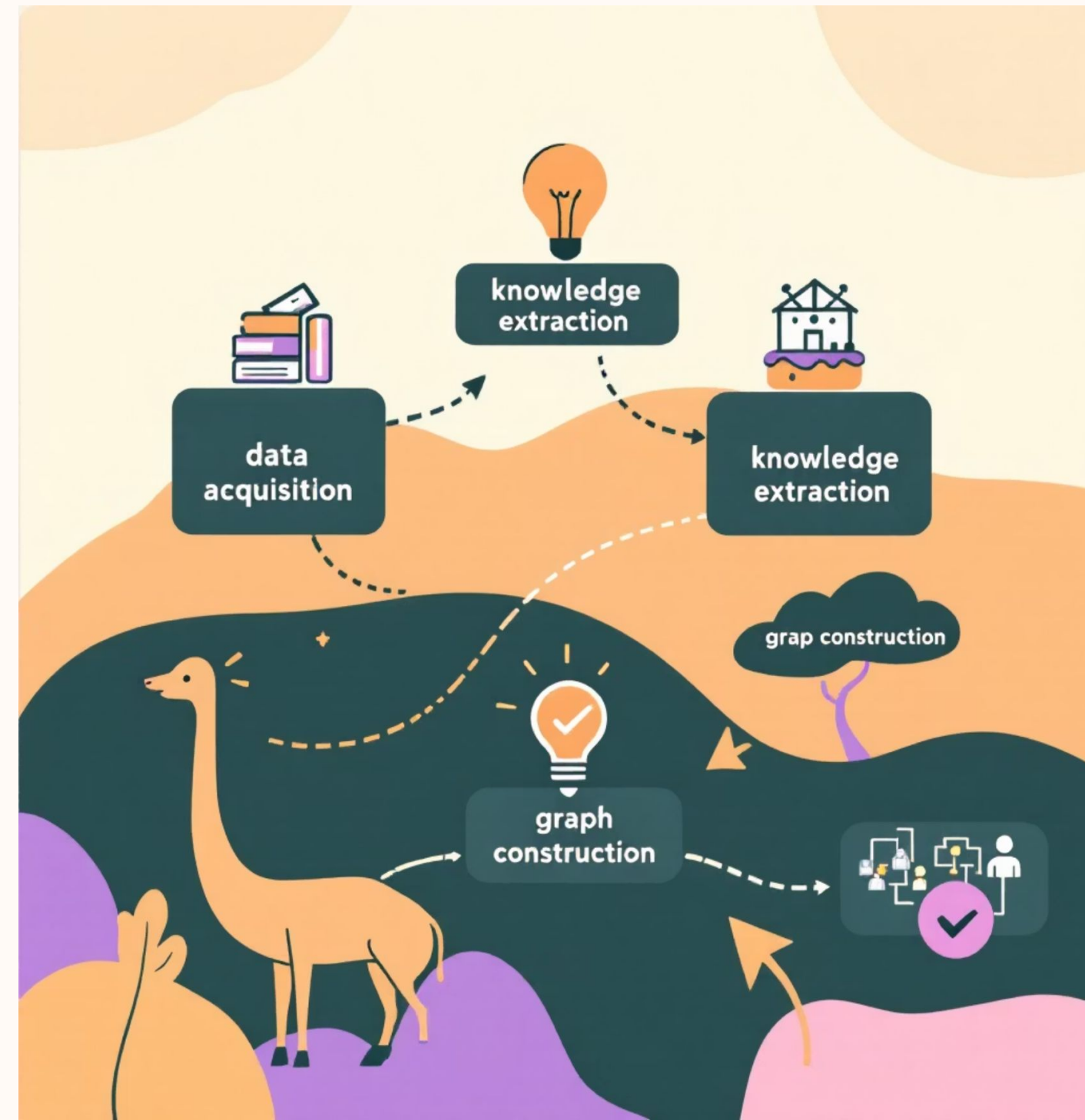
Defining conceptual frameworks and taxonomies

2 Entity Extraction and Alignment

Identifying and aligning entities across data sources

3 Relation Extraction

Discovering semantic relationships between entities

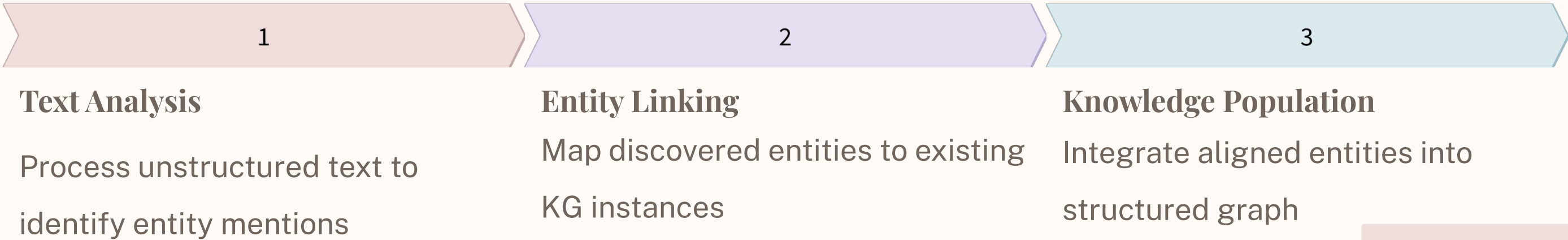


Entity Extraction and Alignment

Primary Objective

Predict and identify the complete set of entities present within textual content for knowledge graph population.

Entity extraction focuses on discovering specific instances mentioned in text and aligning them with existing knowledge graph entities. This critical step populates knowledge graphs with data that maintains ontological consistency and semantic accuracy.



LLMs for Knowledge Graphs

KG Construction Tasks

1 Ontology Generation

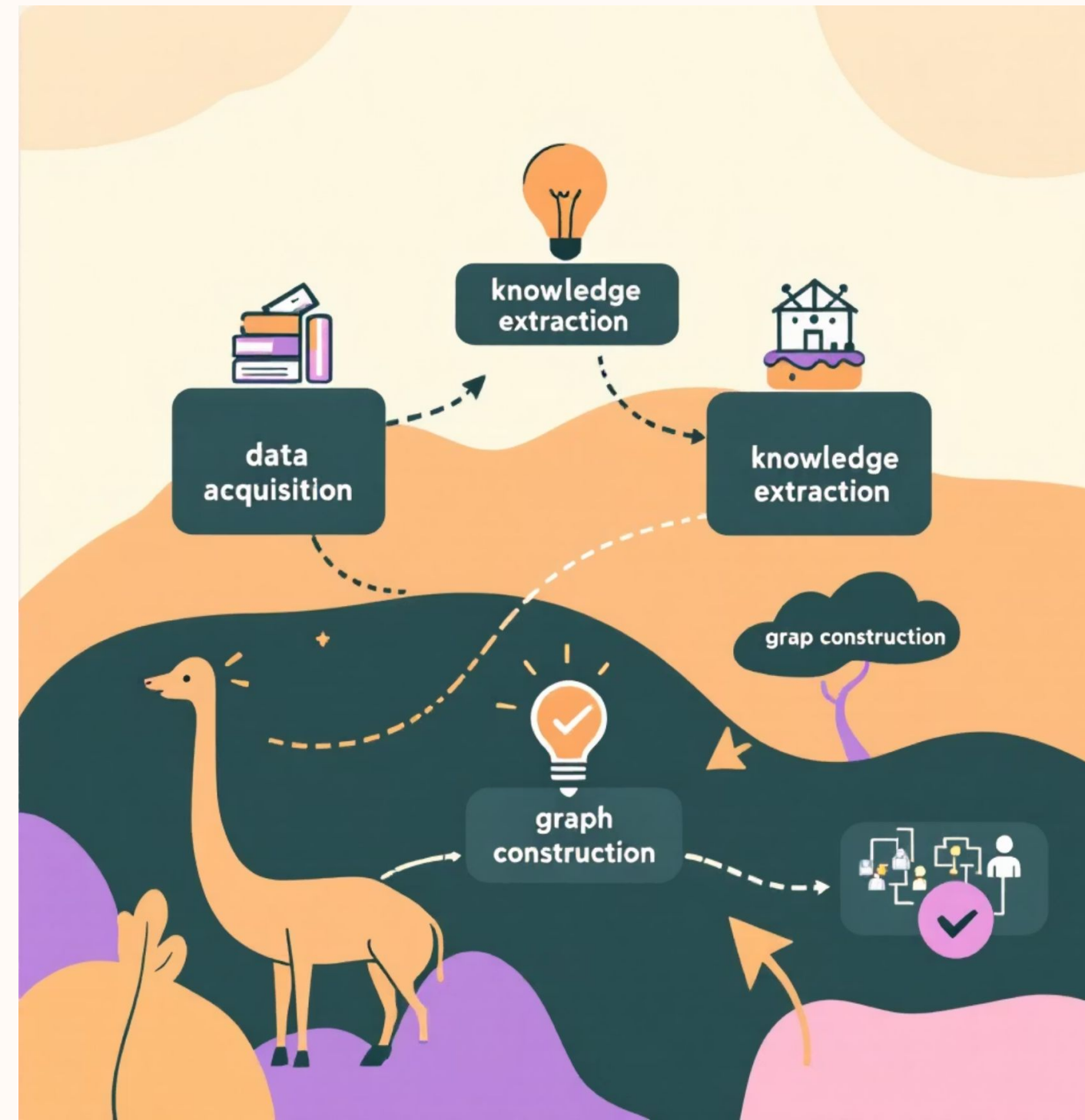
Defining conceptual frameworks and taxonomies

2 Entity Extraction and Alignment

Identifying and aligning entities across data sources

3 Relation Extraction

Discovering semantic relationships between entities



Relation Extraction

Identification and interpretation of semantic connections in textual data through advanced prompting techniques and data generation methods.

Learning Paradigms

Supervised Fine-tuning

Domain-specific adaptation for smaller, specialized models with curated training data

Few-shot Learning

In-context learning with task definitions and example labels for rapid adaptation

Zero-shot Learning

Direct application without task-specific training using Chain-of-Thought prompting



Extract, Define, Canonicalize [1]

1. The process begins with schema-agnostic **Open Information Extraction**. The LLM is instructed to freely extract relational triplets directly from the text.

Result: A "raw," unstandardized triplet is generated, which captures the semantic relationship identified by the model and forms an open KG

2. The LLM is prompted to generate a natural language **definition for the newly extracted relation** (participatedIn), based on the specific context of the sentence. This step is crucial for the subsequent standardization phase.

Result: A clear explanation of the relation's meaning, which will serve as side information for semantic comparison

3. This phase standardizes the raw triplet. Using the generated definition, the system searches for the most semantically similar relation in a target schema (e.g., mission). To prevent over-generalization errors, the LLM is then asked to **verify if the substitution is appropriate** in the given context

Result: Once confirmation is received ("Yes!"), the triplet is converted into its final, standardized, and schema-aligned form

EDC: Extract-Define-Canonicalize

Phase 1: Open Information Extraction



Extract relational triplets from the text: 'Alan Shepard participated in the Apollo 14 mission'

[Alan Shepard, participatedIn, Apollo 14]



Phase 2: Schema Definition



Write a definition for the relation 'participatedIn' in the current context

The subject entity took part in the mission specified by the object entity



Phase 3: Schema Canonicalization



The most semantically similar relation in the schema is 'mission'



Can the relation 'participatedIn' be replaced by 'mission' given the current context?

Yes! The converted relational triplet is [Alan Shepard, mission, Apollo 14]



LLMs for Knowledge Graphs



KG Construction

Automated generation of knowledge graph structures from unstructured text data



KG-to-Text Generation

Converting structured knowledge into natural language descriptions



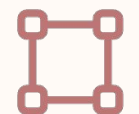
KG Reasoning

Inferring new knowledge from existing graph relationships and patterns



KG Completion

Filling missing entities and relationships in incomplete knowledge graphs



KG Embedding

Creating dense vector representations of entities and relations



KG Validation

Verifying accuracy and consistency of knowledge graph content



KGs-to-Text Generation

Transform structured knowledge graph data into human-readable, natural-language text while preserving semantic accuracy and maintaining important contextual details.

1

Structured KG Data

Entities, relations, and attributes in formal representation

2

LLM Processing

Semantic alignment and natural language generation

3

Natural Text Output

Coherent, contextually accurate descriptions

This approach enables automated documentation generation, knowledge base summarization, and improved human-computer interaction through natural language interfaces.

Example: Drug Information

RDF Input

```
@prefix ex: <http://example.org/>
.ex:Aspirin rdf:type ex:Drug ;
ex:usedToTreat ex:Pain ,
ex:Fever ,
ex:Inflammation ;
ex:hasActiveIngredient
ex:AcetylsalicylicAcid .
```

LLM Prompt

Task: Generate a coherent English paragraph describing the drug entity, combining all RDF information.

Data: Aspirin is a Drug that treats Pain, Fever, and Inflammation. Its active ingredient is Acetylsalicylic Acid.

Generated Output

"Aspirin is a pharmaceutical drug primarily used to treat pain, fever, and inflammation. The medication contains acetylsalicylic acid as its active ingredient, making it effective for various therapeutic applications."

LLMs for Knowledge Graphs



KG Construction

Automated generation of knowledge graph structures from unstructured text data



KG-to-Text Generation

Converting structured knowledge into natural language descriptions



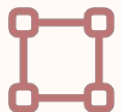
KG Reasoning

Inferring new knowledge from existing graph relationships and patterns



KG Completion

Filling missing entities and relationships in incomplete knowledge graphs



KG Embedding

Creating dense vector representations of entities and relations



KG Validation

Verifying accuracy and consistency of knowledge graph content

KGs Reasoning



Logical Inference

Deriving new facts from existing knowledge using established logical rules and semantic relationships within the graph structure.



Question Answering

Enabling complex multi-hop reasoning to answer queries that require connecting multiple entities and relationships.



Knowledge Completion

Identifying missing links and entities through reasoning about implicit relationships and semantic patterns.



Natural Language Inference Framework

Premise	Hypothesis	Label
A man inspects the uniform of a figure in some East Asian country.	The man is sleeping.	Contradiction
An older and younger man smiling.	Two men are smiling and laughing at cats playing.	Neutral
A soccer game with multiple males playing.	Some men are playing a sport.	Entailment

NLI provides the logical foundation for knowledge graph reasoning by establishing relationships between textual statements and enabling sophisticated inference capabilities.



Reasoning Task Categories

1

Link Prediction

Input: (X, isParentOf, Y), (Y, isParentOf, Z)

Output: (X, isGrandparentOf, Z)

2

Multi-hop Reasoning

Input: (Drug A treats Disease X), (Disease X causes Symptom Y)

Output: (Drug A relieves Symptom Y)

3

Type Inference

Input: (Ibuprofen, treats, Headache), (Headache, rdf:type, Symptom)

Output: (Ibuprofen, treats, Symptom)

4

Contradiction Detection

Input: (Aspirin, rdf:type, Anticoagulant), (Aspirin, rdf:type, NotAnticoagulant)

Output: Logical conflict detected

LLMs for Knowledge Graphs



KG Construction

Automated generation of knowledge graph structures from unstructured text data



KG-to-Text Generation

Converting structured knowledge into natural language descriptions



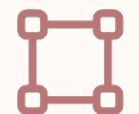
KG Reasoning

Inferring new knowledge from existing graph relationships and patterns



KG Completion

Filling missing entities and relationships in incomplete knowledge graphs



KG Embedding

Creating dense vector representations of entities and relations



KG Validation

Verifying accuracy and consistency of knowledge graph content

Knowledge Graph Completion

The systematic process of identifying and adding missing information to knowledge graphs, enhancing completeness and accuracy through intelligent inference and pattern recognition.



Triple Classification

Validate the accuracy and plausibility of given entity-relation-entity triples using semantic understanding.



Link Prediction

Predict missing components (head entities, tail entities, or relations) in incomplete knowledge triples.



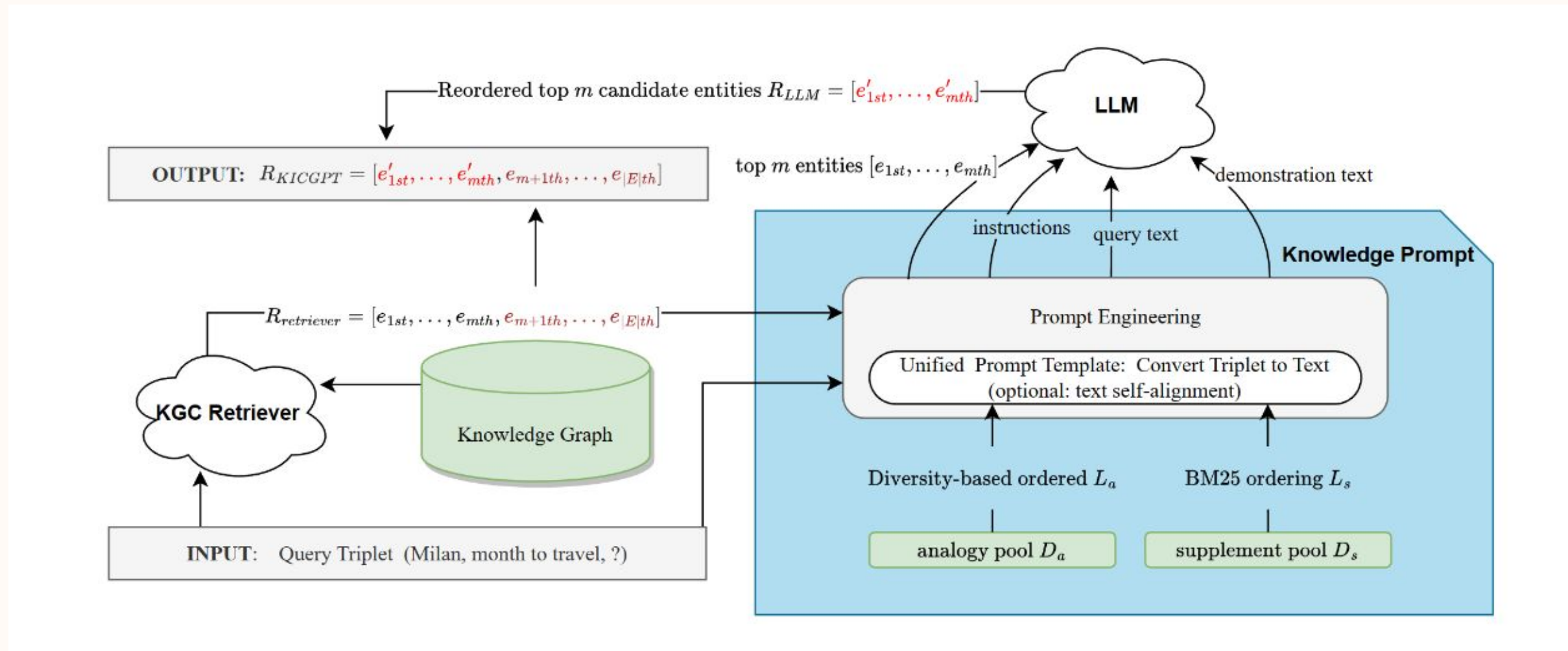
Entity Classification

Determine appropriate types or categories for entities based on their relationships and contextual information.

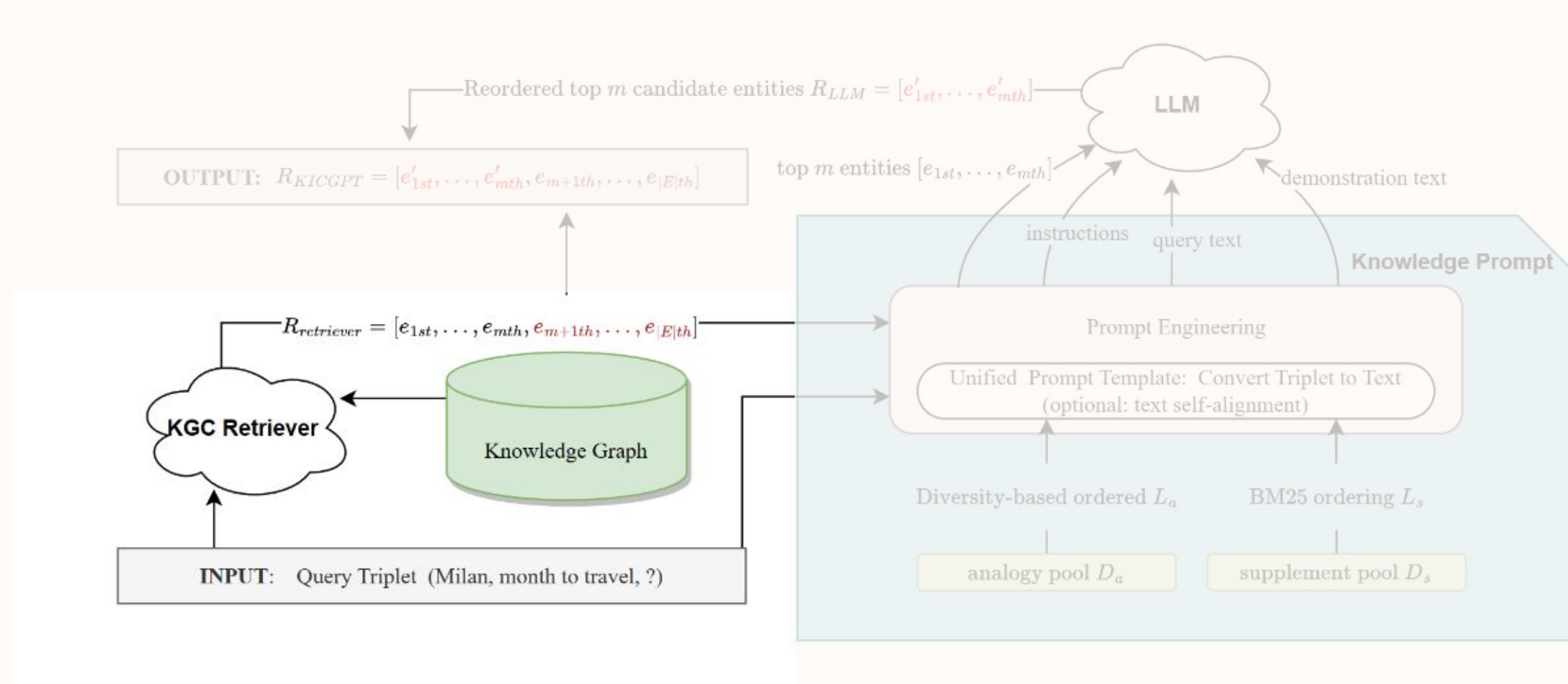


KICGPT [2]

LLM with Knowledge in Context for KG Completion



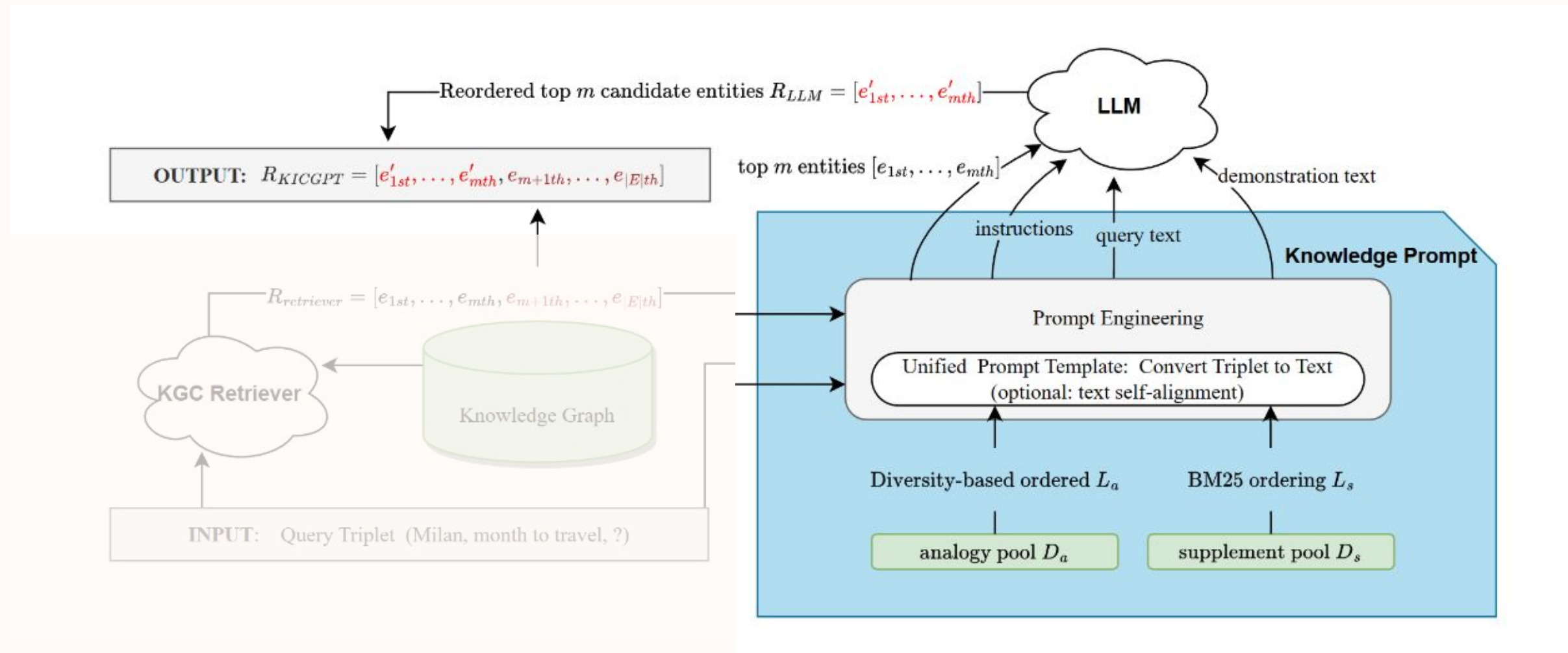
The goal of KICGPT is to integrate a triple-based retriever and a LLM to overcome the limitations of each approach when used alone



Retriever:

This is a traditional, triple-based KGC model (like RotatE). These models work by analyzing only the structure of the knowledge graph (that is, the existing connections between entities).

Given a query (an incomplete triple such as (head entity, relation, ?)), the retriever processes the entire graph and generates an ordered list, $R_{\text{retriever}}$, of all possible candidate entities to complete the triple. This list is ranked from most to least plausible according to the model



LLM:

A large-scale language model, such as ChatGPT.

The LLM does not analyze all entities. Instead, it focuses only on the top m candidates provided by the retriever. Its role is to perform a **re-ranking** of this short-list, leveraging its semantic and contextual knowledge.

To perform this task effectively, the LLM is guided by an in-context learning (ICL) strategy called **Knowledge Prompt**. This technique involves providing the model with examples (called "demonstrations") drawn directly from the knowledge graph to help it understand the context and the specific relation in the query

LLMs for Knowledge Graphs



KG Construction

Automated generation of knowledge graph structures from unstructured text data



KG-to-Text Generation

Converting structured knowledge into natural language descriptions



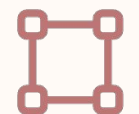
KG Reasoning

Inferring new knowledge from existing graph relationships and patterns



KG Completion

Filling missing entities and relationships in incomplete knowledge graphs



KG Embedding

Creating dense vector representations of entities and relations



KG Validation

Verifying accuracy and consistency of knowledge graph content



KGs Embedding

Knowledge Graph embedding maps KG elements into continuous vector spaces, enabling machine learning models to capture semantic properties and relationships effectively.

Enhanced Task Performance

LLMs can directly perform entity alignment and other KG tasks using learned embeddings

Knowledge Transfer

This allows you to capture properties and relationships in a format that machine learning models can easily process.

LLMs for Knowledge Graphs



KG Construction

Automated generation of knowledge graph structures from unstructured text data



KG-to-Text Generation

Converting structured knowledge into natural language descriptions



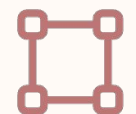
KG Reasoning

Inferring new knowledge from existing graph relationships and patterns



KG Completion

Filling missing entities and relationships in incomplete knowledge graphs



KG Embedding

Creating dense vector representations of entities and relations



KG Validation

Verifying accuracy and consistency of knowledge graph content

KG Validation Techniques

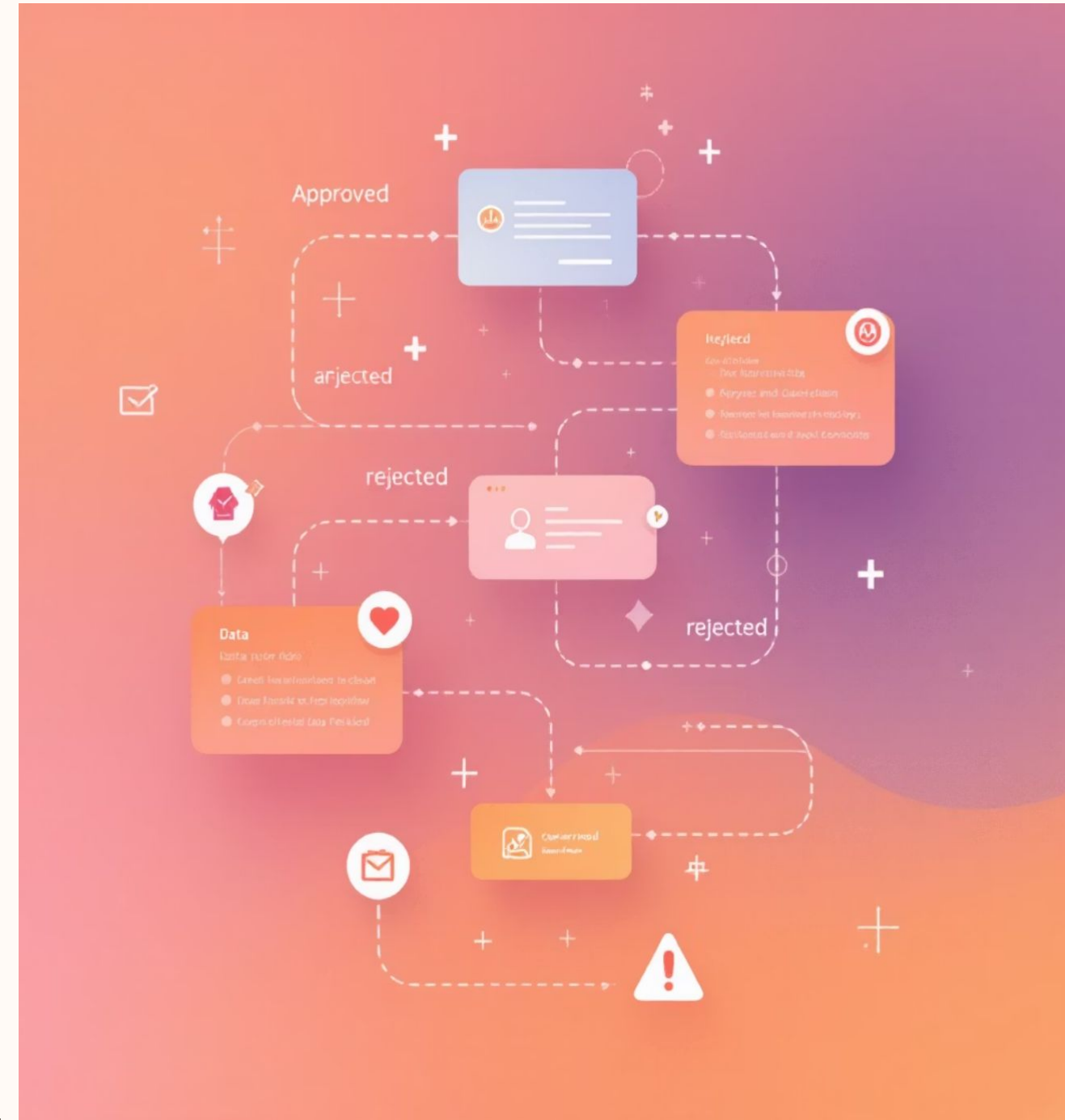
Fact Checking

LLMs analyze triples converted to text format to verify factual correctness and identify misinformation using external knowledge sources.

Inconsistency Detection

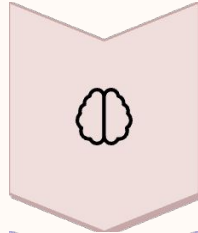
Identifying logical contradictions within KGs by exploiting semantic and structural information to generate validation rules.

These validation approaches ensure knowledge graphs maintain accuracy, consistency, and completeness across diverse domains.



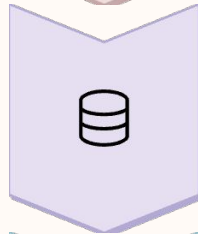
The LLM-KG Synergy

"The synergy between LLMs and KGs opens new frontiers for AI applications, enhancing machines' ability to process, interpret, and generate information with context and accuracy"



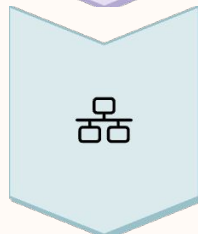
LLMs for KGs

How LLMs bolster and refine Knowledge Graph functionality



KGs-enhanced LLMs

KGs leverage to enhance LLM capabilities and accuracy



LLM-KG Cooperation

Collaborative synergy achieving advanced, complex outcomes





KG-Enhanced LLMs

A synergistic approach where knowledge graphs enhance LLM capabilities, addressing inherent limitations like hallucinations and outdated information.



Core Objective

Generate more accurate, consistent, and contextually relevant responses by leveraging structured knowledge



Problem Resolution

Address LLM limitations including hallucinations, domain knowledge gaps, and temporal information constraints

KG Enhancement Strategies

Knowledge Injection

Integrate KG representations with text embeddings during preprocessing stages

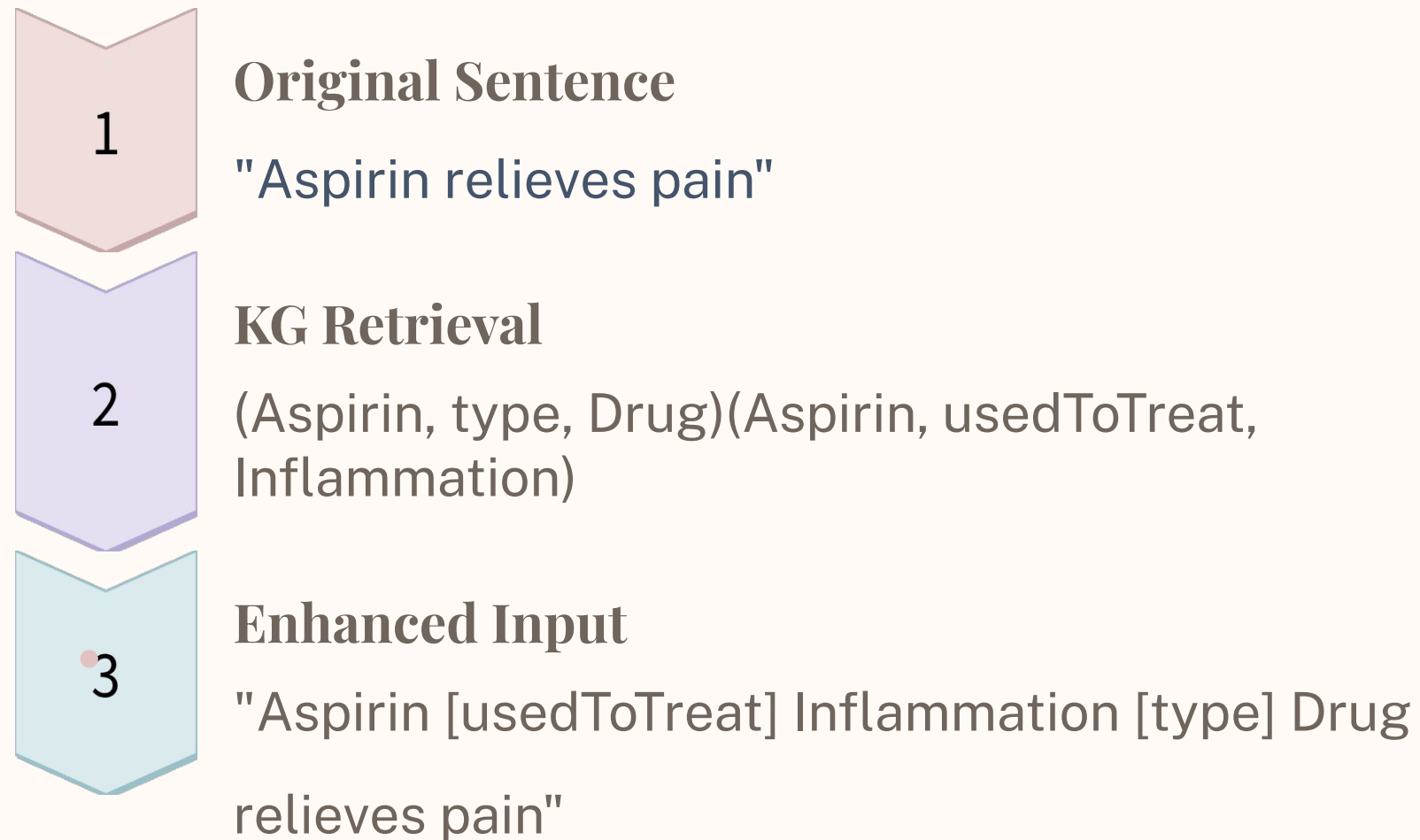
Knowledge injection typically occurs before computing text embeddings, creating enriched contextual representations for improved model performance.

Retrieval Augmented Generation (RAG)

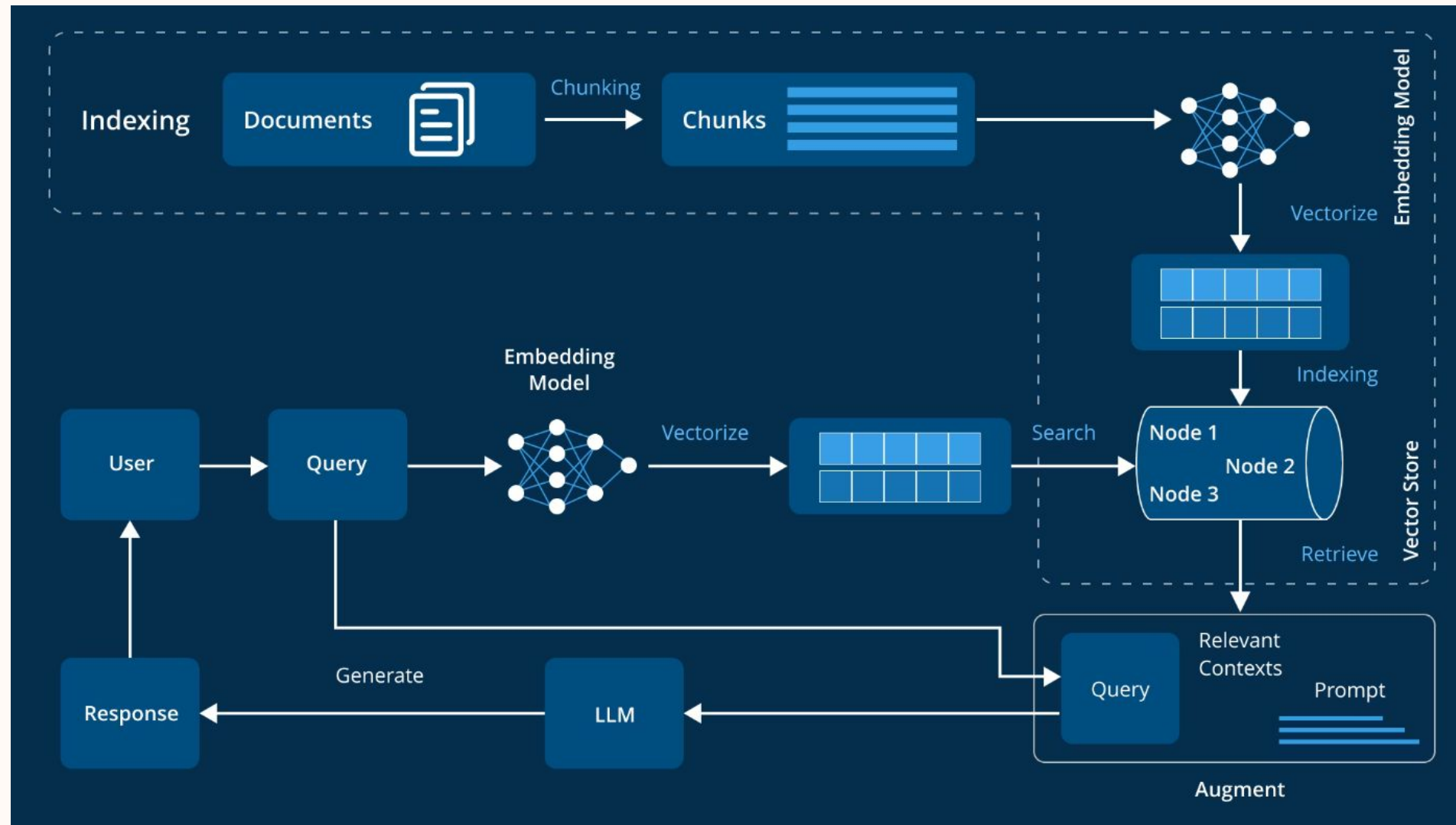
Dynamically retrieve relevant knowledge during generation process



Knowledge Injection Pipeline

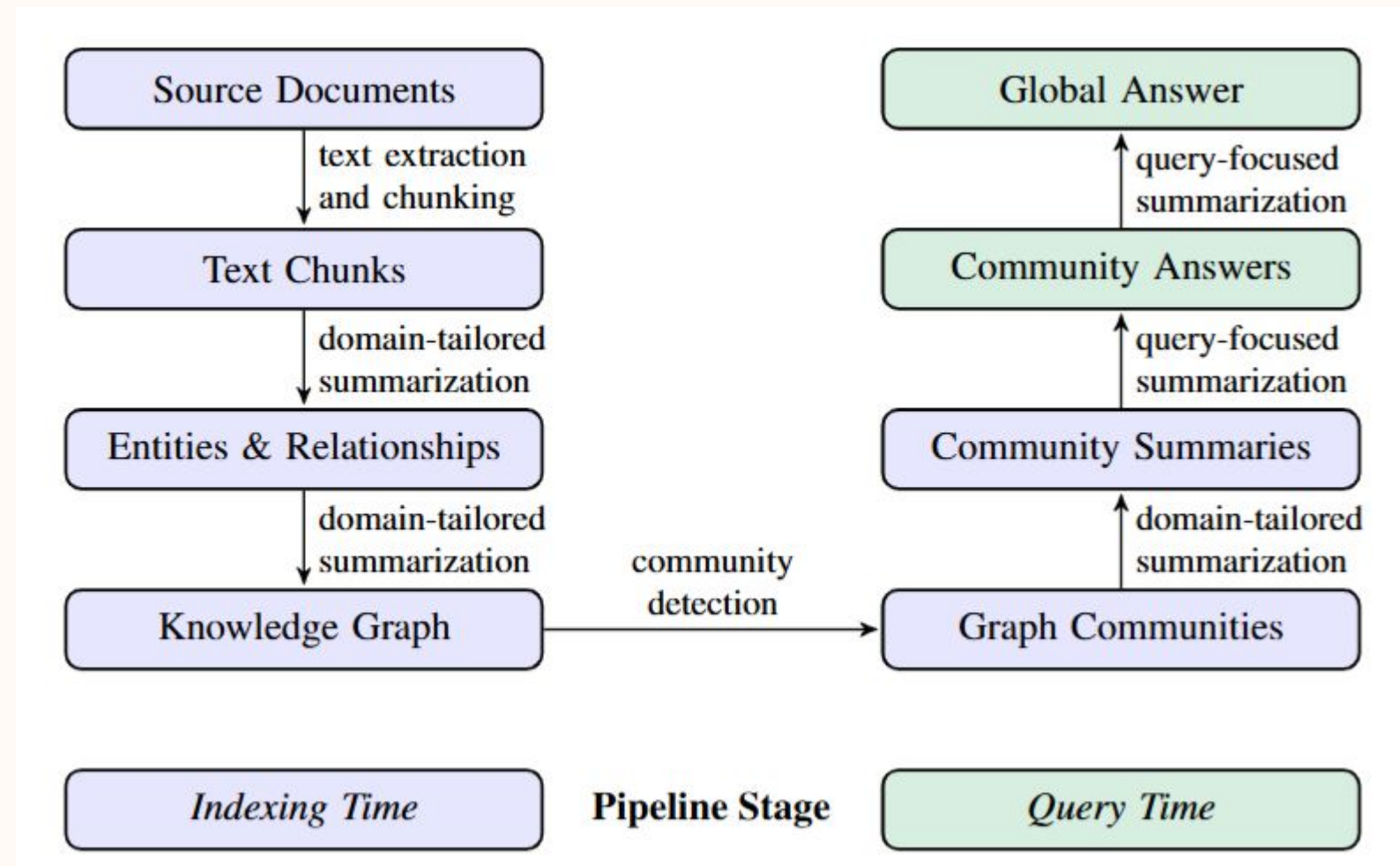


Retrieval Augmented Generation (RAG)



RAG systems combine retrieval mechanisms with generative models, enabling dynamic access to external knowledge sources during text generation.

Graph RAG [3]



Overcome the limitations of traditional vector-based RAG by enabling answers to "**global questions**", those that require an understanding of the entire document corpus, not just a few specific chunks

Graph RAG

1. Entity and Relationship Extraction:

- Source documents are split into text chunks.
- A LLM processes each chunk to **extract key entities** (e.g., people, organizations), **the relationships between them**, and their descriptions. This process acts as a form of abstractive summarization.

2. Knowledge Graph Construction:

- The extracted entities and relationships are aggregated to build a **knowledge graph**. Entities become the nodes and relationships become the edges of the graph.

3. Community Detection:

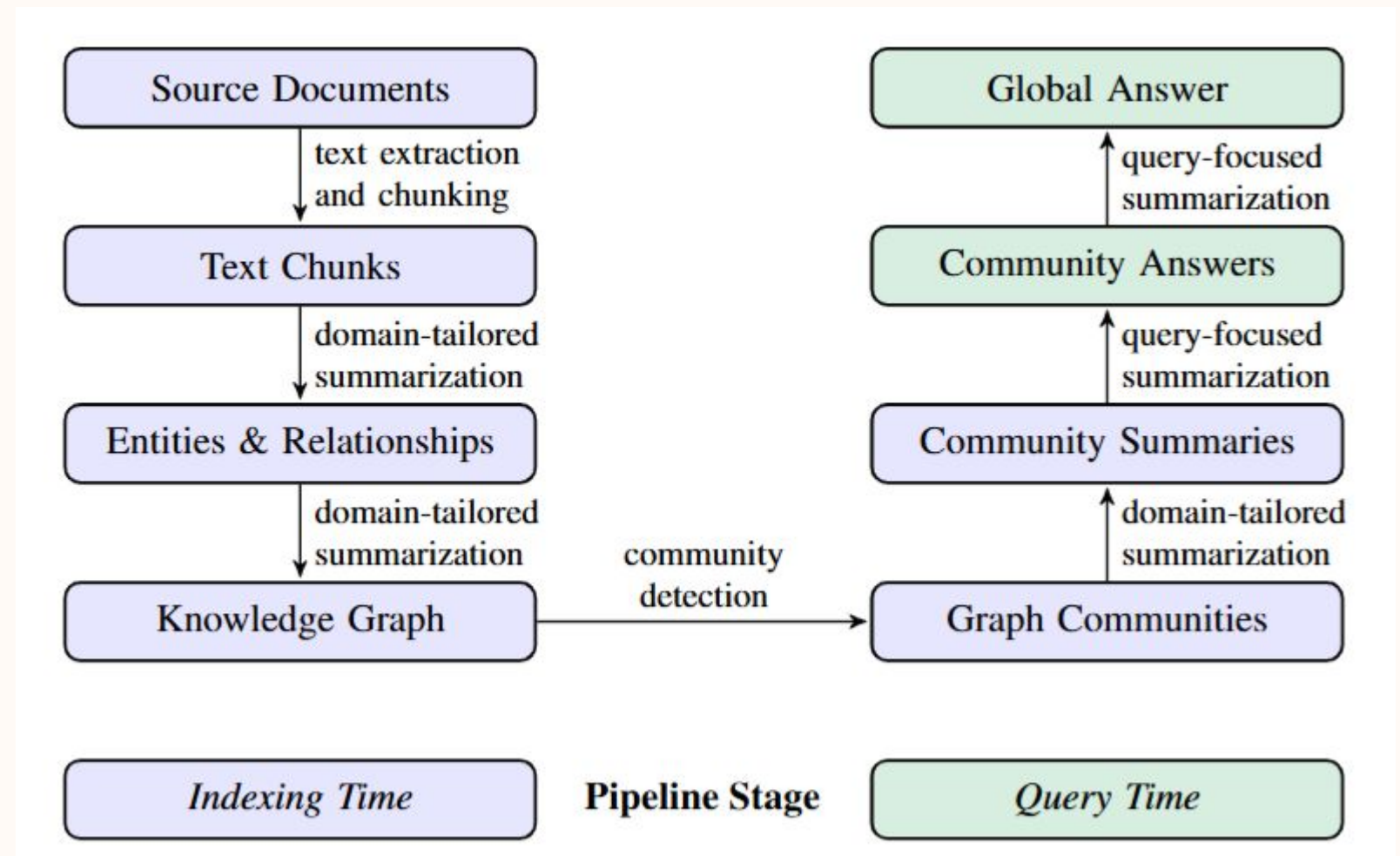
- The graph is partitioned into **hierarchical "communities"** — groups of strongly interconnected nodes (using algorithms like Leiden). This creates a thematic structure for the data.

4. Community Summary Generation:

- For each detected community, the LLM generates a detailed, **report-like summary**. These are created in a bottom-up, hierarchical manner, with summaries from lower-level communities being used to create those for higher-level communities

Phase 1: Graph Index Creation (Indexing Time)

This phase is performed once to prepare the data, making it efficiently queryable.



Graph RAG

Phase 2: Answer Generation (Query Time)

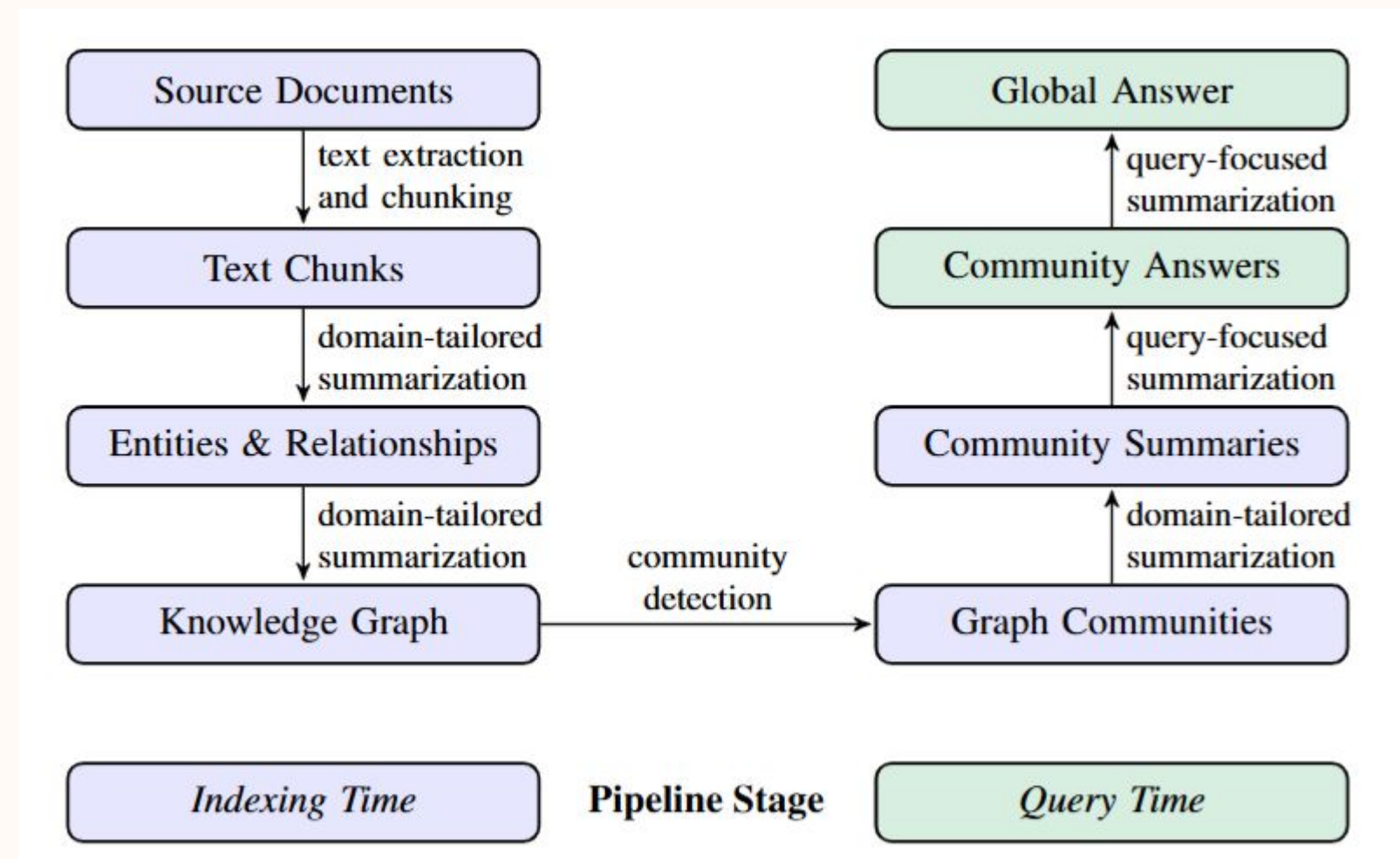
This phase is activated when a user asks a question.

1. Preparation and "Map":

- Given a user query, the system selects pre-generated community summaries from a chosen hierarchical level.
- These summaries are then fed to an LLM to generate multiple **partial answers** in parallel, each receiving a helpfulness score.

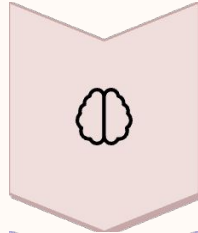
2. "Reduce" to Global Answer:

- The partial answers are sorted by their helpfulness score. The top-ranked ones are combined and used to generate the **final, global answer** that is presented to the user



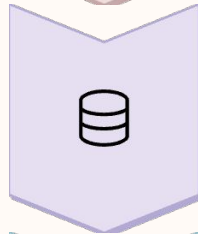
The LLM-KG Synergy

"The synergy between LLMs and KGs opens new frontiers for AI applications, enhancing machines' ability to process, interpret, and generate information with context and accuracy"



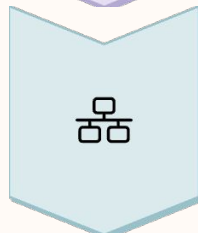
LLMs for KGs

How LLMs bolster and refine Knowledge Graph functionality



KGs-enhanced LLMs

KGs leverage to enhance LLM capabilities and accuracy



LLM-KG Cooperation

Collaborative synergy achieving advanced, complex outcomes



LLM-KG Cooperative Framework

?

Multi-Hop Question Generation

Creating complex queries requiring reasoning across multiple KG relations



Complex Question Answering

Answering multi-step questions through KG traversal and reasoning



Query Generation from Text

Converting natural language into formal SPARQL queries



KG-Powered Chatbots

Conversational agents combining KG lookups with fluent responses

This cooperative approach enables sophisticated reasoning capabilities that leverage both structured knowledge and natural language understanding.

Cooperative Techniques Overview

Technique	Description	Example
Multi-hop Question Generation	Generate complex questions requiring reasoning over multiple KG relations	<i>"Which drugs treat conditions caused by inflammation?"</i>
Multi-hop Question Answering	Answer questions by reasoning across 2+ KG triples	<i>Q: "Which diseases treated by aspirin are genetic?" → (aspirin → treats → X → type → genetic)</i>
Query Generation from Text	Convert natural language into formal SPARQL queries	<i>"Show authors born in Italy" → SELECT ?author WHERE { ?author dbo:birthPlace dbr:Italy }</i>
Querying LLMs with SPARQL	Retrieve triples from KG via SPARQL and inject into LLM prompts	<i>Prompt: "Given that (Aspirin, treats, Pain), what does Aspirin treat?"</i>
KG-powered Chatbots	Conversational agents combining KG lookups with fluent LLM responses	<i>User: "What does ibuprofen do?" → KG query + answer synthesis by LLM</i>

These techniques demonstrate the practical implementation of LLM-KG cooperation for enhanced reasoning and knowledge retrieval capabilities.

Implementation Examples

1

Multi-hop Question Generation

Prompt: Given the triples [(Inflammation, causes, Pain), (Aspirin, treats, Inflammation)], generate a question requiring multi-hop reasoning.
→ "Which medication treats pain by reducing inflammation?"

2

Multi-hop Question Answering

Prompt: KG facts: (Aspirin, treats, Inflammation), (Inflammation, causes, Pain). Q: How does Aspirin help with pain?
→ "Aspirin treats inflammation, which causes pain."

3

Query Generation

Prompt: "Which authors were born in Italy?" → SPARQL query generation.
→ `SELECT ?author WHERE { ?author dbo:birthPlace dbr:Italy }`

4

Querying LLMs with SPARQL

Prompt: Facts: (Aspirin, treats, Inflammation), (Inflammation, causes, Pain). Q: Why is Aspirin effective for pain?
→ Answer using facts in fluent language.

5

KG Chatbots

Prompt: User: "What diseases can Ibuprofen treat?" KG: (Ibuprofen, treats, Fever), (Ibuprofen, treats, Headache)
→ "Ibuprofen is commonly used to treat fever and headaches."